

A New Operational Transformation Framework for Real-Time Group Editors

Rui Li, *Student Member, IEEE*, and Du Li, *Member, IEEE*

Abstract—Group editors allow a group of distributed human users to edit a shared multimedia document at the same time over a computer network. Consistency control in this environment must not only guarantee convergence of replicated data, but also attempt to preserve intentions of operations. Operational transformation (OT) is a well-established method for optimistic consistency control in this context and has drawn continuing research attention since 1989. However, counterexamples to previous works have often been identified despite the significant progress made on this topic over the past 15 years. This paper analyzes the root of correctness problems in OT and establishes a novel operational transformation framework for developing OT algorithms and proving their correctness.

Index Terms—Consistency control, group editors, groupware, operational transformation.

1 INTRODUCTION

GROUP editors are a classic model and research vehicle for distributed interactive groupware applications because they typically manipulate shared data in a coordinated manner [5]. Operational transformation (OT) has been well accepted in group editors for achieving optimistic consistency control [6], [19]. OT allows local operations to execute in a nonblocking manner to achieve high local responsiveness and unconstrained collaboration. Remote operations are *transformed* before they are executed such that inconsistencies are repaired.

Despite the significant progress that has been achieved over the past 15 years, a notable fact in the history of OT is that the discovery and solution of various OT puzzles (i.e., correctness problems in previous OT algorithms) have been a main driver of research [10], [17], [19]. In our opinion, however, the existence of OT puzzles can be largely attributed to the lack of a suitable theoretical framework for guiding the design and verification of OT algorithms. More specifically, the well-established frameworks [14], [16], [17], [20] rely on conditions that are difficult to verify in practice and do not address how to develop correct OT algorithms.

In this paper, we propose a novel OT framework to address this weakness of previous work. Based on a concept called “operation effects relation,” we define two criteria, causality preservation and operation effects relation preservation, for verifying the correctness of OT algorithms. Our framework comes with a practicable approach to developing and proving OT algorithms. In this approach, the sufficient conditions for transformation functions are first identified and a special transformation path is chosen to satisfy those conditions. Thus, the correctness of the whole algorithm is guaranteed.

The remainder of this paper is organized as follows: Section 2 gives background on OT algorithms and motivates this research. Section 3 formalizes the operation effects relation and a consistency model. Section 4 establishes the sufficient conditions of transformation functions. Section 5 shows how to choose the right transformation paths. Section 6 gives an example of the algorithm. Section 7 summarizes the contributions and future directions of this work.

2 BACKGROUND AND MOTIVATION

2.1 Background on Group Editors and OT

Following the established conventions [6], we model the shared data as a linear string s . Let the position of the first character in any nonempty string be zero. Assume that every appearance of any character has a different object id. Note that this assumption only serves analysis purposes and we do not really need object ids for characters in actual implementation. In this paper, a “character” refers to the object that carries the character, whose ASCII code is possibly only one of its attributes. If c is the i th character in any nonempty string s , we say $s[c] = i$ and $c = s[i]$. For convenience, we also use s to denote the set of characters in s .

For simplicity, we only consider two primitive operations: $\text{ins}(p, c)$, which inserts a character c at position p , and $\text{del}(p)$, which deletes the character at position p . Apparently, the position parameter p of any operation o is defined relative to some state s . If o is generated in s , then s is called its *generation state*. If o is executed in s , then s is called its *execution state*. Due to concurrency, an operation’s execution state is not necessarily equivalent to its generation state if the operation is executed at a remote site [19].

Given operation o , function $\text{pos}(o)$ returns its position value, $\text{type}(o)$ returns its operation type (*ins* or *del*), $\text{char}(o)$ returns the effect character to be inserted or deleted, $\text{id}(o)$ returns the id of the site that generates o , $\text{gst}(o)$ denotes its generation state, and $\text{est}(o)$ denotes its execution state. The fact that the execution of o in state s yields s' is denoted by $\text{exec}(s, o) = s'$. As in [6], [13], [20], we use two long-established relations without further definition: For any

• The authors are with the Department of Computer Science, Texas A&M University, College Station, Texas 77843-3112.
E-mail: rui@neo.tamu.edu, lidu@cs.tamu.edu.

Manuscript received 17 Jan. 2005; revised 12 Aug. 2005; accepted 23 Mar. 2006; published online 25 Jan. 2007.

Recommended for acceptance by P. McKinley.

For information on obtaining reprints of this article, please send e-mail to: tpds@computer.org, and reference IEEECS Log Number TPDS-0036-0105.

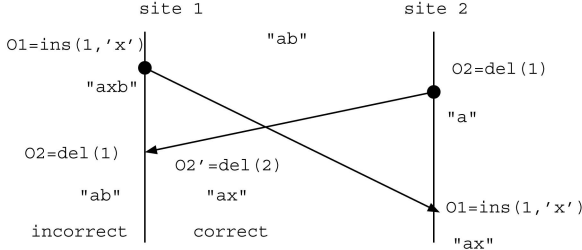


Fig. 1. OT transforms o_2 and then executes o_2' .

two operations o_1 and o_2 , $o_1 \rightarrow o_2$ iff o_1 happened before o_2 and $o_1 \parallel o_2$ iff o_1 is concurrent with o_2 .

The basic idea of OT is to execute any local operation as soon as it is generated for high local responsiveness. Remote operations are *transformed* against concurrent operations that have been executed locally before its execution. A history buffer HB is maintained at each site to keep track of all executed operations in their order of execution.

As a simple example, consider the scenario in Fig. 1. Suppose two sites start from the same initial state $s_1^0 = s_2^0 = \text{"ab."}$ Site 1 performs $o_1 = \text{ins}(1, 'x')$ to insert character 'x' before 'b', yielding $s_1^1 = \text{exec}(s_1^0, o_1) = \text{"axb,"}$ while site 2 concurrently performs $o_2 = \text{del}(1)$ to delete character 'b', yielding $s_2^1 = \text{exec}(s_2^0, o_2) = \text{"a."}$ When $o_2 = \text{del}(1)$ arrives at site 1, if it is executed as it is, then the wrong character 'x' will be deleted. This is because o_2 is generated in s_2^0 without the knowledge of o_1 , but its execution state s_1^1 has been changed by the execution of o_1 , which invalidates its position parameter. The intuition of OT [6] is to shift the position of o_2 to *incorporate* the effect of o_1 such that the result o_2' can be correctly executed in state s_1^1 . This process is called *inclusion transformation* (IT) [20].

Because a character has been inserted by o_1 on the left of its intended position, o_2 should delete the character currently at position 2 instead of 1, i.e., $o_2' = \text{IT}(o_2, o_1) = \text{del}(2)$. The execution of o_2' in s_1^1 leads to the correct state $s_1^2 = \text{"ax,"}$ which is identical to the final state at site 2 after o_2 and o_1 are executed in tandem. As a result, OT seems able to achieve convergence and preserve intentions of operations despite the different orders of execution at different sites.

Another type of transformation function is called *exclusion transformation* (ET) [20]. In the above example, given o_1 defined in s_1^0 and o_2' defined in $s_1^1 = \text{exec}(s_1^0, o_1)$, $\text{ET}(o_2', o_1)$ excludes the effect of o_1 from o_2' as if o_1 had not been executed in s_1^0 . The result $o_2 = \text{ET}(o_2', o_1) = \text{del}(1)$ is exactly the execution form of o_2 as defined relative to s_1^0 .

It has been generally accepted [19] that each OT algorithm consists of two parts: a set of *transformation functions* (such as IT and ET) that determine how one operation is transformed against another and a *control procedure* that determines how an operation is transformed against a given operation sequence (e.g., the history buffer). The control procedure is also responsible for generating and propagating local operations as well as executing remote operations.

2.2 Convergence in Group Editors

Most existing consistency control methods, e.g., [1], [3], [23], [24], are motivated in traditional noninteractive applications

in which user interface effects are not interesting. Hence, convergence often suffices and can be achieved by enforcing a total order of writes. However, the example in Fig. 1 reveals that serialization may violate the operation intentions, e.g., when o_2 is executed after o_1 as it is.

The general assumption underlying interactive groupware applications, such as group editors, is that users are aware of the changes made by collaborators and are able to discover and resolve semantic conflicts (e.g., grammatical errors) in a timely manner [6], [7], [20]. Hence, the OT algorithms generally do not consider reads. Writes are often distinguished as two primitive operations, insert and delete, for creating new objects and removing existing objects. This distinction of writes makes it possible to achieve a high level of concurrency while reducing the chance of operation conflicts and overwriting.

The main purpose of Internet-based productivity applications, such as group editors, is to promote the productivity of human users as a group. It is accepted in this context that high local responsiveness and high concurrency are conducive to individual and group productivity [6], [20]. Traditional consistency control methods such as locking and serialization [3] generally sacrifice responsiveness and concurrency when they are pessimistic and may cause the loss of interaction results when optimistic. OT seems a promising consistency control method that is able to achieve high responsiveness, high concurrency, and convergence while preserving interaction results and operation intentions [20].

Preserving the interaction results of all collaborators does not necessarily lead to a chaotic system. The results can be presented to users, e.g., in different colors and multiple versions [21], which is essentially a user interface problem and beyond the scope of this paper. However, the system should do as much work as it can to reduce the cognitive overheads of users. That is, the data replicas must eventually converge (after all generated operations are executed at all sites) and the converged final state must be somehow constrained, e.g., by preserving causality and operation intentions [20].

2.3 Previous OT Frameworks

A framework consists of a theory, which defines a consistency model of the system, and a technical approach, which guides the development of algorithms that fulfill the model. Two OT frameworks have been proposed in the literature.

The **first** OT framework is established in Ressel et al. [14]. In the theory part, it formalizes two consistency criteria, causality preservation and convergence (CC). In the approach part, it proves that any OT-based algorithm can achieve convergence in the presence of arbitrary transformation paths if its IT function satisfies two properties, as defined below:

Definition 1. Given three concurrent operations o_1 , o_2 , and o_3 that are defined in the same state s , let $o_1' = \text{IT}(o_1, o_2)$ and $o_2' = \text{IT}(o_2, o_1)$. TP1 and TP2 are defined as follows:

$$\begin{aligned} \text{TP1: } \text{exec}(s, [o_1, o_2']) &= \text{exec}(s, [o_2, o_1']) \\ \text{TP2: } \text{IT}(\text{IT}(o_3, o_1), o_2') &= \text{IT}(\text{IT}(o_3, o_2), o_1'). \end{aligned}$$

TP1 originally appears in [6] and TP2 is first discussed in [8]. TP1 ensures that, if $o_1 \parallel o_2$, the effect of executing o_1

before o_2 is the same as executing o_2 before o_1 . TP2 ensures that transforming any operation o_3 along different paths, $[o_1, o_2]$ and $[o_2, o_1]$, will yield the same result. These two properties ensure that arbitrary transformation paths can lead to a consistent final state. In other words, it is not necessary to enforce a global total order between operations in order to achieve convergence because divergence can always be repaired by operational transformation.

The CC framework has influenced most of the OT algorithms developed since it was proposed in 1996, including adOPTed [14], GOT [20], GOTO [19], [17], [4], NICE [15], SOCT [16], [22], TIBOT [12], and SDT [9], [10], as they more or less struggle on the TP2 condition. Influential as it is, this framework is flawed in the following ways: First, the consistency model does not explicitly constrain convergence. Hence, it is *theoretically* acceptable in the model that the system always converges in an empty state no matter what the users do. Second, in the technical approach part, TP1 and TP2 are only for constraining IT. As revealed later [16], [17], [19], [20], ET is conceptually an inverse of IT and indispensable for implementing group undo and building transformation paths. However, the CC framework does not constrain ET. Additionally, it does not provide guidelines on how to develop IT functions that satisfy TP2. In fact, TP2 had never been verified until in our recent work [9], [10], [11].

The **second** framework is established in Sun et al. [20] with three consistency criteria: causality preservation, convergence, and intention preservation (CCI). The intention preservation condition is the first attempt in the literature to explicitly constrain convergence in interactive groupware. Due to its intuitiveness, this model has been well-accepted in group editors, such as [2], [4], [16], [18], [22]. However, Sun et al. did not clearly formalize exactly what is intention-preserving and what is not in the original work [20] and its main follow-up [17]. Hence, it is difficult to evaluate the correctness of OT algorithms under the CCI framework.

Example 1. A well-known controversial scenario follows:

Suppose three sites start from the same initial state “abc”. Concurrently, site 1 performs $o_1 = \text{ins}(2, 'x')$ to insert ‘x’ between ‘b’ and ‘c’, site 2 performs $o_2 = \text{ins}(1, 'y')$ to insert ‘y’ between ‘a’ and ‘b’, and site 3 performs $o_3 = \text{del}(1)$ to delete ‘b.’ Intuitively, the only correct final result must be “ayxc.” However, result “axyc” is also referred to as intention-preserving in [20].

In the technical approach part of [17], the CCI framework inherits the TP1 and TP2 constraints on IT, while still not providing guidelines for developing such IT functions. It also defines several constraints that relate IT and ET, e.g., reversibility and transpose properties. However, ET is treated as a second-class citizen and no standalone constraints are defined. As an important contribution, Sun et al. [20], [17] define the precondition of $\text{IT}(o_1, o_2)$ as $\text{est}(o_1) = \text{est}(o_2)$ and the precondition of $\text{ET}(o_1, o_2)$ as $\text{est}(o_1) = \text{exec}(\text{est}(o_2), o_2)$. However, these two conditions are not really sufficient, as will be illustrated later in Examples 3 and 4 in Section 4.

3 THE CONSISTENCY MODEL

As revealed in Example 1, the main problem of the established CCI model [17], [20] is that it only considers the “intentions” or effects of single operations in their generation states while failing to capture a global picture of the whole system. Conversely, if a global picture was present when transforming the two operations, o_1 and o_2 , we would know how to correctly determine the relation between their effect characters, ‘x’ and ‘y.’

We address this problem by introducing a new concept of *effects relation* $\prec$ to denote the order between characters. In Example 1, we have ‘b’ $\prec$ ‘x’ $\prec$ ‘c’ when o_1 is generated and ‘a’ $\prec$ ‘y’ $\prec$ ‘b’ when o_2 is generated. Then, by transitivity, we infer ‘y’ $\prec$ ‘x’ based on which we can always correctly transform o_1 and o_2 . In this way, we can model both effects of single operations and the relation between effects of concurrent operations.

Our early result, as presented in [10], [11], introduced but did not rigorously formalize the concept of effects relation. Additionally, it was developed under the influence of CCI and still strived to satisfy TP1 and TP2. Nevertheless, the TP2 proofs in [11] are very complicated although they only treat two characterwise primitives. How to scale the proofs to handle more sophisticated string operations [20] is still an open problem. Related works in [12], [15], [20], [22] free TP2 by enforcing a unique transformation path. However, they are developed under the CCI framework and must verify the condition of intention preservation as required in CCI.

The purposes of the present paper are two-fold: First, we will present a rigorous definition of the effects relation. Second, we will explore a more practicable approach that no longer requires the verification of TP1 and TP2.

3.1 Effects Relation

Suppose there is an observer to the system (group editor) who observes the progress (every operation execution) at each site. A virtual global data structure Ω is maintained to indicate the effects relation $\prec$. Ω is initialized by the initial system state s^0 and incrementally updated by the execution of operations. Before we define rules for initializing and updating Ω , we study the order in which Ω is updated.

Note that the same operation is invoked to update Ω multiple times because it is executed once at every site in the system. Hence, we have to examine these invocations at the event level. An allowed updating path is a sequence of operations (invocations), $o_1, o_2, \dots, o_n$, in which, for any two operations o_i and o_j , o_i precedes (or is invoked before) o_j , $1 \leq i < j \leq n$, iff one of the following conditions holds: 1) o_i is executed earlier than o_j at the same site, 2) o_i is the local execution and o_j is a remote execution of the same operation, or 3) there exists another execution o_k such that o_i precedes o_k and o_k precedes o_j .

We annotate Ω with a superscript to denote its incremental construction process. Ω^0 contains the initial effects relation: For any two characters $c_i, c_j \in s^0$, we add pair $\langle c_i, c_j \rangle$ or $c_i \prec c_j$ into Ω^0 iff $s^0[c_i] > s^0[c_j]$. Given any updating path $P = [o_1, o_2, \dots, o_n]$, the execution of o_i updates Ω^{i-1} to Ω^i , where $1 \leq i \leq n$. For convenience, we assume that there are two invisible characters: c_b and c_e in any state s such that $s[c_b] = -1$ and $s[c_e] = |s|$ and, for any visible character $c \in s$, we have $c_b \prec c \prec c_e$.

Assume the execution of any operation o in its generation state s is “correct.” If $o = \text{ins}(p, c)$, it inserts a new character c between two existing characters $s[p-1]$ and $s[p]$. Hence, we add two pairs $\langle s[p-1], c \rangle$ and $\langle c, s[p] \rangle$ into Ω . On the other hand, if $o = \text{del}(p)$, it deletes the existing character $s[p]$. Its execution does not change the ordering between existing characters. Hence, all pairs in Ω carry over.

Obviously, Ω^0 defines a total order over all characters in s^0 . We would like Ω to be maintained as a total order over all characters that have appeared in the system upon every update. To achieve this, we require that 1) every execution of a deletion deletes the character it intended at its generation state and 2) only insertions at their generation states introduce new pairs while any remote execution of an insertion does not introduce pairs that contradict existing pairs in Ω . Therefore, we only need to consider the local execution of insertions when we define the effects relation.

Definition 2 (Effects Relation). Assume Ω^0 is initialized from the system initial state s^0 by adding $c_i \prec c_j$ into Ω^0 for any c_i and c_j such that $s^0[c_i] < s^0[c_j]$. Given an updating path P , assume Ω^{j-1} is a total order produced by the first $j-1$ executions in P . Now, we consider how the j th execution o_j extends Ω^{j-1} into Ω^j . Without loss of generality, assume $o_j = \text{ins}(p_j, c_j)$ is an insertion to be executed in its generation state s . Suppose c_j is to be inserted between $\beta = s[p_j-1]$ and $\alpha = s[p_j]$. Let Σ be the set of characters that appear in Ω^{j-1} . Let $B = \{\beta\} \cup \{c \in \Sigma | c \prec \beta\}$, $A = \{\alpha\} \cup \{c \in \Sigma | \alpha \prec c\}$, and $D = \Sigma - A - B = \{c \in \Sigma | \beta \prec c \prec \alpha\}$. For any character $c_i \in \Sigma$, we decide the relation between c_i and c_j by the following rules and add the new relation to Ω^j :

1. if $c_i \in B$, then $c_i \prec c_j$;
2. if $c_i \in A$, then $c_j \prec c_i$;
3. if $c_i \in D$ and $c_i \in s^0$, we mandate $c_j \prec c_i$;
4. if $c_i \in D$ and $c_i \notin s^0$, assuming c_i is originally inserted by o_i and $o_i \rightarrow o_j$, we mandate $c_j \prec c_i$; and
5. if $c_i \in D$ and $c_i \notin s^0$, assuming c_i is originally inserted by o_i and $o_i \parallel o_j$, we first (recursively) derive the relation between c_j and every character $c \in D$, where $c \in s^0$ or c is originally inserted by some o such that $o \rightarrow o_i$ or $o \rightarrow o_j$, and then consider the following rules:
 - a. if there exists c in D such that either $c_i \prec c \prec c_j$ or $c_j \prec c \prec c_i$ is implied, then we add $c_i \prec c_j$ or $c_j \prec c_i$ accordingly;
 - b. if no such relation can be inferred, we mandate $c_i \prec c_j$ if $\text{id}(o_i) < \text{id}(o_j)$, or $c_j \prec c_i$ if otherwise.

In the above definition, rules 1 and 2 are straightforward. Rules 3 and 4 are to mandate an order between c_j and those that have been deleted before o_j is generated in state s . Rule 5 conceptually breaks ties between two characters that are inserted concurrently between the same two characters. It appears complicated because the ties cannot be safely broken by simply comparing their site ids directly, as is illustrated by the following example:

Example 2. Suppose three sites start from $s^0 = \text{“ab.”}$ By definition, Ω is initialized to include $\text{‘a’} \prec \text{‘b.’}$ Site 1 generates $o_1 = \text{ins}(1, \text{‘x’})$ to get “axb” and, concurrently, Site 2 generates $o_2 = \text{ins}(1, \text{‘y’})$ to get “ayb” . After

executing o_1 , Site 3 generates $o_3 = \text{ins}(1, \text{‘z’})$ to get “azxb” . We have $o_1 \parallel o_2$, $o_1 \rightarrow o_3$ and $o_2 \parallel o_3$. Suppose an updating path is $[o_1, o_3, o_2]$. The question is how to determine the order between ‘y’ and ‘z’ and the order between ‘y’ and ‘x’ . If we directly compared their site ids, we would get $\text{‘y’} \prec \text{‘z’}$ and $\text{‘x’} \prec \text{‘y’}$, which violates the established order of $\text{‘z’} \prec \text{‘x’}$. By Rule 5, to determine the order between ‘y’ and ‘z’ , we first determine the order between ‘y’ and ‘x’ , which is $\text{‘x’} \prec \text{‘y’}$ by Rule 5b. Then, by transitivity (Rule 5a), we infer $\text{‘z’} \prec \text{‘x’} \prec \text{‘y’}$. Hence, the final result must be “azxyb.”

Theorem 1. The effects relation $\prec$ is a total order.

Definition 3 (Landmark Characters). For any two characters c_i and c_j that are originally inserted by o_i and o_j , respectively, character c is a landmark character between them iff 1) $c_i \prec c \prec c_j$ or $c_j \prec c \prec c_i$ and 2) $c \in s^0$ or the operation that originally inserted c happened before at least one of o_i and o_j . The set of the landmark characters between c_i and c_j is denoted by $C_{ld}(c_i, c_j)$.

For example, in the scenario of Example 1, let $c_1 = \text{‘x’}$ and $c_2 = \text{‘y’}$. By Definition 2, the set of landmark characters between them is $C_{ld}(c_1, c_2) = \{\text{‘b’}\}$. In some updating paths, such as $[o_3, o_1, o_2]$, when o_2 is transformed with o_1 , the landmark character ‘b’ is not present in the current state. This causes tie-breaking problems in previous work such as GOTO [19], which will be further explained in Example 3.

The concept of landmark character will be used in our correctness analyses. The reason we exclude operations that are concurrent with, or are generated after, both o_i and o_j is that, due to causality, the effects relation between c_i and c_j should not depend on the effects of those operations. By the following lemma, landmark characters will play an important role in determining the effects relation of operations:

Lemma 1. Suppose that two characters, c_i and c_j , are originally inserted by o_i and o_j , respectively, and that o_i precedes o_j in an updating path and $c_i \notin \text{gst}(o_j)$. If $C_{ld}(c_i, c_j) = \emptyset$, then we only need Rules 4 and 5b to determine the relation between c_i and c_j .

3.2 Correctness Criteria

A quiescent state in a group editor means all generated operations have been executed at all sites [6]. System consistency requires not only that all sites have the same set of objects (characters), but also that all these objects are ordered the same way. Based on the above notion of effects relation, we define the following correctness conditions:

Definition 4 (Correctness Criteria). A group editor is correct if it always maintains the following two properties:

1. Causality preservation: For any two operations o_1 and o_2 , if $o_1 \rightarrow o_2$, then o_1 is executed before o_2 at all sites.
2. Effects relation preservation: The effects relation is preserved at every operation execution.

Causality preservation is a fundamental criterion in typical distributed systems as well as group editors [6], [14], [16], [20]. The effects relation preservation as a system-specific measure constrains convergence in a group editor

and its behavior. It is easy to show that a correct group editor converges in any quiescent state because all sites have the same set of characters that are ordered in the same way.

Corollary 1. *Any correct group editor converges in quiescent states.*

In the following, we further study how to constrain all the possible states and operation executions in a group editor:

Definition 5 (Reachable States). *Any state s is reachable iff, for any $c_1, c_2 \in s$, if $s[c_1] > s[c_2]$, then $c_1 \prec c_2$.*

Conceptually, a reachable state is such that the ordering of its characters is consistent with the effects relation.

Definition 6 (Admissible Operations). *Given an operation o that is defined in a reachable state s , i.e., $s = \text{est}(o)$, we say o is admissible in s iff one of the following conditions holds: 1) $\text{type}(o) = \text{del}$ and the execution of o in s only leads $\text{char}(o)$ to be deleted, 2) $\text{type}(o) = \text{ins}$ and $s = \text{gst}(o)$, or 3) $\text{type}(o) = \text{ins}$, $s \neq \text{gst}(o)$, and for any $c \in s$: $\text{pos}(o) \leq s[c]$ iff $\text{char}(o) \prec c$ or $s[c] < \text{pos}(o)$ iff $c \prec \text{char}(o)$.*

Intuitively, when executing a deletion, the system should not delete any character that is not intended by the original operation; when executing an insertion, the new character ordering should not violate the effects relation definition. As an example of admissible operations, consider the execution of o_1 and o_2 in Fig. 1. At site 1, if $o_2 = \text{del}(2)$ is executed, it is not admissible because it targets the wrong character. At site 2, if $o_1 = \text{ins}(0, 'x')$ is executed instead, then 'x' is inserted at the wrong position, which violates the effects relation established by o_1 when it is generated.

Lemma 2. *If o is admissible in a reachable state s , then $s' = \text{exec}(s, o)$ is reachable.*

Lemma 3. *Given an admissible operation o in a reachable state s , if $o = \text{del}(p)$, we have $\text{char}(o) = s[p]$, or if $o = \text{ins}(p, c)$, we have $s[i] \prec c \prec s[j]$ for any i and j such that $-1 \leq i < p$ and $p \leq j \leq |s|$. Recall that $s[c_0] = -1$ and $s[c_e] = |s|$.*

Based on the concept of admissible operations, we may further simplify the verification of system correctness by the following lemma:

Lemma 4. *If every operation a system executes is admissible, the system preserves the effect relation.*

4 TRANSFORMATION FUNCTIONS AND CONDITIONS

The concept of effects relation lays a theoretical foundation for analyzing system behavior. However, it relies on a global data structure that is expensive to maintain in practice. Here, we explore an approach that only uses local information. More specifically, each site maintains a local history buffer HB for recording operations in their execution order and a local table ER for recording the derived effects relation. The effects relation is derived only by information in HB and ER .

We use operation keys to uniquely identify operations. The key of any operation o , denoted by function $\text{key}(o)$, is a pair of the id of its generation site $\text{id}(o)$ and its sequence

number at that site. Due to the established state vector maintenance protocol [6], [20], no two different operations share the same key value. Unlike $\text{pos}(o)$, the value of $\text{key}(o)$ does not change. Since each operation has a unique effect character, $\text{char}(o)$ can be safely replaced by $\text{key}(o)$ in algorithm implementation. The relation ER is implemented as a hash table, which stores pairs of operation keys: We infer $\text{char}(o_x) \prec \text{char}(o_y)$ if pair $\langle \text{key}(o_x), \text{key}(o_y) \rangle$ is found in ER , or equivalently, $\langle \text{char}(o_x), \text{char}(o_y) \rangle \in ER$.

In this section, we define the IT/ET functions based on relation $\prec$ and study sufficient conditions under which relation $\prec$ can be correctly determined using only local information stored in ER and HB and the basic operation parameters $\text{pos}(o)$, $\text{id}(o)$, and $\text{type}(o)$.

4.1 Inclusion Transformation and Correctness

The purpose of $o'_1 = \text{IT}(o_1, o_2)$, where $s = \text{est}(o_1) = \text{est}(o_2)$ and $s' = \text{exec}(s, o_2)$, is to incorporate the effect of o_2 into o_1 such that the result o'_1 can be correctly executed in state s' . As defined in Algorithm 1, we first call Algorithm 2 (get_er_IT) to determine the effects relation of o_1 and o_2 and then use this relation to decide how to compute o'_1 .

Algorithm 1 $\text{IT}(o_1, o_2)$: o'_1

```

1:  $\text{get\_er\_IT}(o_1, o_2)$ 
2: if  $\text{char}(o_1) = \text{char}(o_2)$  then
3:   return  $o'_1 \leftarrow \phi$  //  $\phi$  means identity operation
4: else
5:    $o'_1 \leftarrow o_1$ 
6:   if  $\text{char}(o_2) \prec \text{char}(o_1)$  then
7:     if  $\text{type}(o_2) = \text{ins}$  then
8:        $\text{pos}(o'_1) \leftarrow \text{pos}(o_1) + 1$ 
9:     else //  $\text{type}(o_2) = \text{del}$ 
10:       $\text{pos}(o'_1) \leftarrow \text{pos}(o_1) - 1$ 
11:   end if
12: end if
13: return  $o'_1$ 
14: end if
```

If $\text{char}(o_1) \prec \text{char}(o_2)$, it means that $\text{char}(o_2)$ is on the right side of $\text{char}(o_1)$ and the execution of o_2 does not affect o_1 . Hence, o_1 is returned as it is. In the case of $\text{char}(o_2) \prec \text{char}(o_1)$, however, if o_2 inserted a character on the left of $\text{char}(o_1)$, the position of o_1 should be increased by one, or if o_2 deleted a character on the left, the position of o_1 should be decreased by one.

Assuming that every insertion introduces a new character, it is impossible that o_1 has the same effect character as another operation o_2 . However, two concurrent deletions may intend to delete the same existing character. To handle this situation, we adopt a policy in which this character is only deleted once. The deletion received later is transformed into an **identity operation** ϕ , which will not be executed. This corresponds to lines 1-2 in Algorithm 1 and lines 8-9 in Algorithm 2.

Algorithm 2 $\text{get_er_IT}(o_1, o_2)$: relation of $\text{char}(o_1)$ and $\text{char}(o_2)$

```

1: return the relation if it is found in  $ER$ 
2:  $er \leftarrow \text{char}(o_2) \prec \text{char}(o_1)$ 
3: if  $\text{pos}(o_1) < \text{pos}(o_2)$  then
```

```

4:   $er \leftarrow \text{char}(o_1) \prec \text{char}(o_2)$ 
5:  else if  $\text{pos}(o_1) = \text{pos}(o_2)$  then
6:    if  $\text{type}(o_1) = \text{type}(o_2) = \text{ins} \wedge \text{id}(o_1) < \text{id}(o_2)$  then
7:       $er \leftarrow \text{char}(o_1) \prec \text{char}(o_2)$ 
8:    else if  $\text{type}(o_1) = \text{type}(o_2) = \text{del}$  then
9:       $er \leftarrow \text{char}(o_1) = \text{char}(o_2)$ 
10:   else if  $\text{type}(o_1) = \text{ins} \wedge \text{type}(o_2) = \text{del}$  then
11:      $er \leftarrow \text{char}(o_1) \prec \text{char}(o_2)$ 
12:   end if
13: end if
14: record  $er$  in  $ER$ 
15: return  $er$ 

```

We define Algorithm 2 for determining the effects relation between any two given operations. It returns the relation if it has already been recorded in ER . Otherwise, it derives the relation by a set of rules and records the result in ER . In the following lemmas, we study by cases the sufficient conditions of $\text{get_er_IT}(o_1, o_2)$ or, more specifically, the rules in lines 2-13. They all assume that state $s = \text{est}(o_1) = \text{est}(o_2)$ is reachable and o_1, o_2 are admissible in s . We say $\text{get_er_IT}(o_1, o_2)$ is correct iff the relation between $\text{char}(o_1)$ and $\text{char}(o_2)$ as determined by its rules is identical to the effects relation obtained by Definition 2.

It is well-accepted that the two operations in IT be defined relative to the same state, i.e., $\text{est}(o_1) = \text{est}(o_2)$, for, otherwise, it does not make sense to compare their position parameters [16], [20]. However, condition $\text{est}(o_1) = \text{est}(o_2)$ alone is not sufficient for correctly determining the relation between $\text{char}(o_1)$ and $\text{char}(o_2)$ in IT, as is shown in the following scenario:

Example 3. In Example 1, consider how operations o_2 and o_1 are executed at Site 3 by (the IT and control procedure of) GOTO [19]. Suppose o_2 is received first. We get $o'_2 = \text{IT}(o_2, o_3) = \text{ins}(1, 'y')$ and the history buffer is $[o_3, o'_2]$. When o_1 arrives, we first get $o'_1 = \text{IT}(o_1, o_3) = \text{ins}(1, 'x')$ and then perform $o''_1 = \text{IT}(o'_1, o'_2)$. Because their positions tie, if we naively compare their site ids to break the tie, we get $o'_1 = \text{ins}(1, 'x')$ and the final state “axyc”, which is wrong due to our analysis in Example 1. In this scenario, the precondition $\text{est}(o'_1) = \text{est}(o'_2)$ is satisfied, but the system results in the wrong state and violates the correct effects relation in which ‘y’ should precede ‘x’.

Lemma 5. $\text{get_er_IT}(o_1, o_2)$ can correctly determine the effects relation between o_1 and o_2 if $\text{type}(o_1) = \text{type}(o_2) = \text{ins}$, $o_1 \parallel o_2$, and $(\text{pos}(o_1) \neq \text{pos}(o_2) \text{ or } (\text{pos}(o_1) = \text{pos}(o_2) \wedge C_{ld}(\text{char}(o_1), \text{char}(o_2)) = \emptyset))$.

Proof. If $\text{pos}(o_1) \neq \text{pos}(o_2)$, then there must exist at least one character c between $\text{pos}(o_1)$ and $\text{pos}(o_2)$ in s . That is, $C_{ld}(\text{char}(o_1), \text{char}(o_2)) \neq \emptyset$. Without loss of generality, by Lemma 3, suppose $\text{char}(o_1) \prec c \prec \text{char}(o_2)$, $\text{pos}(o_1) < \text{pos}(o_2)$, and $\text{pos}(o_1) \leq s[c] < \text{pos}(o_2)$. By $\text{get_er_IT}()$, we derive $\text{char}(o_1) \prec \text{char}(o_2)$, which is consistent with Definition 2.

If $\text{pos}(o_1) = \text{pos}(o_2)$, there is no landmark character between $\text{char}(o_1)$ and $\text{char}(o_2)$ in state s . Suppose there is an execution path in which o_2 precedes o_1 and all operations preceding o_2 happened before o_1 or o_2 or both.

Since $C_{ld}(\text{char}(o_1), \text{char}(o_2)) = \emptyset$, from Definition 2, we can deduce the relation between $\text{char}(o_1)$ and $\text{char}(o_2)$ by comparing site ids. From $\text{get_er_IT}()$, if $\text{pos}(o_1) = \text{pos}(o_2)$ in state s , we break the tie by comparing site ids, the result of which is consistent with that by Definition 2. $\square$

Lemma 6. $\text{get_er_IT}(o_1, o_2)$ is correct if

$$\text{type}(o_1) = \text{type}(o_2) = \text{del}.$$

Proof. Since both o_1, o_2 are admissible deletions, $\text{char}(o_1)$ and $\text{char}(o_2)$ must be both present in s . Hence, by Lemma 3, we can determine their relation by comparing $\text{pos}(o_1) = s[\text{char}(o_1)]$ and $\text{pos}(o_2) = s[\text{char}(o_2)]$. In particular, if $\text{pos}(o_1) = \text{pos}(o_2)$, they must be referencing the same character, i.e., $\text{char}(o_1) = \text{char}(o_2)$. $\square$

Lemma 7. $\text{get_er_IT}(o_1, o_2)$ is correct if $\text{type}(o_1) = \text{ins}$, $\text{type}(o_2) = \text{del}$, and $\text{char}(o_1) \neq \text{char}(o_2)$.

Lemma 8. $\text{get_er_IT}(o_1, o_2)$ is correct if $\text{type}(o_1) = \text{del}$, $\text{type}(o_2) = \text{ins}$, and $\text{char}(o_1) \neq \text{char}(o_2)$.

Corollary 2. Given that o_1 and o_2 are admissible in a reachable state $s = \text{est}(o_1) = \text{est}(o_2)$, then IT is correct or $o'_1 = \text{IT}(o_1, o_2)$ is admissible in state $s' = \text{exec}(s, o_2)$ if the effects relation between $\text{char}(o_1)$ and $\text{char}(o_2)$ can be found in ER or can be correctly determined by the rules of $\text{get_er_IT}(o_1, o_2)$.

4.2 Exclusion Transformation and Correctness

The purpose of $o'_2 = \text{ET}(o_2, o_1)$, where

$$\text{est}(o_2) = \text{exec}(\text{est}(o_1), o_1),$$

is to exclude the effect of o_1 from o_2 such that $\text{est}(o'_2) = \text{est}(o_1)$ and o'_2 is admissible in $\text{est}(o_1)$, as if o_1 had not been executed. As in Algorithm 3, we first call Algorithm 4 to determine the effect relation between o_1 and o_2 and then use this relation to compute o'_2 .

Algorithm 3 $\text{ET}(o_2, o_1): o'_2$

```

1:  $\text{get\_er\_ET}(o_1, o_2)$ 
2: if  $\text{char}(o_2) = \text{char}(o_1)$  then
3:   halt
4: else
5:    $o'_2 \leftarrow o_2$ 
6:   if  $\text{char}(o_1) \prec \text{char}(o_2)$  then
7:     if  $\text{type}(o_1) = \text{ins}$  then
8:        $\text{pos}(o'_2) \leftarrow \text{pos}(o_2) - 1$ 
9:     else { //  $\text{type}(o_1) = \text{del}$  }
10:       $\text{pos}(o'_2) \leftarrow \text{pos}(o_2) + 1$ 
11:     end if
12:   end if
13: return  $o'_2$ 
14: end if

```

If $\text{char}(o_1)$ and $\text{char}(o_2)$ are the same, meaning that o_2 is to delete the character inserted by o_1 , it does not make sense if o_1 had not been executed before o_2 due to causality. In this case, the system is halted. This corresponds to lines 13-14 in Algorithm 4 and lines 2-3 in Algorithm 3.

If $\text{char}(o_2) \prec \text{char}(o_1)$, meaning that $\text{char}(o_1)$ is on the right side of $\text{char}(o_2)$, then excluding the effect of o_1 does not impact o_2 . Hence, o_2 is returned as it is. If $\text{char}(o_1) \prec \text{char}(o_2)$, however, the position parameter of o_2 should be decreased

by one had o_1 not inserted $\text{char}(o_1)$ on its left, or the position parameter of o_2 should be increased by one had o_1 not deleted on its left.

In Algorithm 4, we define rules for determining the relation of $\text{char}(o_2)$ and $\text{char}(o_1)$ in ET only by their position parameters. Previous work [19], [20] proposed that the precondition of ET be $\text{est}(o_2) = \text{exec}(\text{est}(o_1), o_1)$. However, this condition alone is not sufficient for determining the relation of $\text{char}(o_1)$ and $\text{char}(o_2)$ in ET, as shown in the following scenario:

Example 4. Suppose three sites start from the same state $s^0 = \text{"abc"}$. Site 1 generates $o_1 = \text{del}(1)$ to yield "ac" . Concurrently, site 2 generates $o_2 = \text{ins}(2, \text{'x'})$ to yield "abxc" . At site 3, assume o_1 is executed first, yielding $s_3^1 = \text{exec}(s^0, o_1) = \text{"ac"}$. After that, o_2 is received. We have $o_2' = \text{IT}(o_2, o_1) = \text{ins}(1, \text{'x'})$ and $s_3^2 = \text{exec}(s_3^1, o_2') = \text{"axc"}$. Then, site 3 generates $o_3 = \text{ins}(2, \text{'y'})$, yielding $s_3^3 = \text{"axyc"}$. By Definition 2, we know $\text{char}(o_1) \prec \text{char}(o_2) \prec \text{char}(o_3)$, namely, $\text{char}(o_1) \prec \text{char}(o_3)$. Now, examine how the effects of o_2' and o_1 are excluded from o_3 in turn. First, process $o_3' = \text{ET}(o_3, o_2')$. According to $\text{get_er_ET}()$, we infer $\text{char}(o_2) \prec \text{char}(o_3)$ because $\text{pos}(o_2') < \text{pos}(o_3)$. Hence, $o_3' = \text{ins}(1, \text{'y'})$. Next, consider $o_3'' = \text{ET}(o_3', o_1)$. According to $\text{get_er_ET}()$, we have $\text{char}(o_3) \prec \text{char}(o_1)$ because their positions tie, which contradicts the correct relation $\text{char}(o_1) \prec \text{char}(o_3)$.

Algorithm 4 $\text{get_er_ET}(o_1, o_2)$: relation of $\text{char}(o_1)$ and $\text{char}(o_2)$

```

1: return the relation if it is found in  $ER$ 
2: if  $\text{pos}(o_1) < \text{pos}(o_2)$  then
3:    $er \leftarrow \text{char}(o_1) \prec \text{char}(o_2)$ 
4: else if  $\text{pos}(o_1) > \text{pos}(o_2)$  then
5:    $er \leftarrow \text{char}(o_2) \prec \text{char}(o_1)$ 
6: else { //  $\text{pos}(o_1) = \text{pos}(o_2)$  }
7:   if  $\text{type}(o_1) = \text{type}(o_2) = \text{ins}$  then
8:      $er \leftarrow \text{char}(o_2) \prec \text{char}(o_1)$ 
9:   else if  $\text{type}(o_1) = \text{type}(o_2) = \text{del}$  then
10:     $er \leftarrow \text{char}(o_1) \prec \text{char}(o_2)$ 
11:   else if  $\text{type}(o_1) = \text{del} \wedge \text{type}(o_2) = \text{ins}$  then
12:      $er \leftarrow \text{char}(o_2) \prec \text{char}(o_1)$ 
13:   else { //  $\text{type}(o_1) = \text{ins} \wedge \text{type}(o_2) = \text{del}$  }
14:      $er \leftarrow \text{char}(o_1) = \text{char}(o_2)$ 
15:   end if
16: end if
17: record  $er$  in  $ER$ 
18: return  $er$ 

```

In the following lemmas, we examine by cases the sufficient conditions of $\text{get_er_ET}(o_1, o_2)$ or, more specifically, the rules defined in lines 2-16. They all require the following conditions: $\text{est}(o_2) = \text{exec}(\text{est}(o_1), o_1)$, $\text{est}(o_1)$ and $\text{est}(o_2)$ are reachable states, and o_1 and o_2 are admissible. We say $\text{get_er_ET}(o_1, o_2)$ is correct iff the relation between $\text{char}(o_1)$ and $\text{char}(o_2)$ as determined by its rules is identical to the effects relation obtained by Definition 2.

Lemma 9. $\text{get_er_ET}(o_1, o_2)$ is correct if $\text{type}(o_1) = \text{type}(o_2) = \text{ins}$ and $o_1 \rightarrow o_2$.

Proof. Since o_1 and o_2 are admissible and o_1 is executed before o_2 , the character $\text{char}(o_1)$ must be present in state $s = \text{exec}(\text{est}(o_1), o_1)$. Due to Lemma 3, by comparing their positions $\text{pos}(o_1) = s[\text{char}(o_1)]$ and $\text{pos}(o_2)$, we can correctly infer their effects relation. Specifically, we get $\text{char}(o_2) \prec \text{char}(o_1)$ if $\text{pos}(o_2) \leq \text{pos}(o_1)$ or $\text{char}(o_1) \prec \text{char}(o_2)$ if otherwise. $\square$

Lemma 10. $\text{get_er_ET}(o_1, o_2)$ is correct if $\text{type}(o_1) = \text{ins}$, $\text{type}(o_2) = \text{del}$, and $o_1 \rightarrow o_2$.

Proof. By the given conditions and Lemma 3, $\text{char}(o_1)$ must be present in $s = \text{est}(o_2)$ and $\text{pos}(o_1) = s[\text{char}(o_1)]$. Since o_2 is a deletion and it is admissible in s , it must be that $\text{pos}(o_2) = s[\text{char}(o_2)]$. Hence, we can correctly determine their effects relation by comparing their position parameters. In particular, if $\text{pos}(o_1) = \text{pos}(o_2)$, it must be that $\text{char}(o_1) = \text{char}(o_2)$. $\square$

Lemma 11. $\text{get_er_ET}(o_1, o_2)$ is correct if

$$\text{type}(o_1) = \text{type}(o_2) = \text{del},$$

$$\text{char}(o_1) \neq \text{char}(o_2), \text{ and } o_1 \rightarrow o_2.$$

Lemma 12. $\text{get_er_ET}(o_1, o_2)$ is correct if $\text{type}(o_1) = \text{del}$, $\text{type}(o_2) = \text{ins}$, $\text{char}(o_1) \neq \text{char}(o_2)$, $o_1 \rightarrow o_2$, and

$$(\text{pos}(o_1) \neq \text{pos}(o_2) \text{ or } \text{pos}(o_1) = \text{pos}(o_2) \text{ and } C_{ld}(\text{char}(o_1), \text{char}(o_2)) = \emptyset).$$

Proof. Let $s^1 = \text{est}(o_1)$ and $s^2 = \text{est}(o_2)$. Since o_1 is a deletion admissible in s^1 , by Lemma 3, it must be $\text{pos}(o_1) = s^1[\text{char}(o_1)]$. If $\text{pos}(o_1) < \text{pos}(o_2)$, there must be at least one character between $\text{pos}(o_1)$ and $\text{pos}(o_2)$ in s^2 . Hence, we infer $\text{char}(o_1) \prec \text{char}(o_2)$. If $\text{pos}(o_1) > \text{pos}(o_2)$, similarly, we infer $\text{char}(o_2) \prec \text{char}(o_1)$.

In the case that $\text{pos}(o_1) = \text{pos}(o_2)$, by Lemma 3, it must be that o_2 inserts a character between the two characters $s^2[\text{pos}(o_2) - 1] = s^1[\text{pos}(o_1) - 1]$ and

$$s^2[\text{pos}(o_2)] = s^1[\text{pos}(o_1) + 1].$$

By Definition 2, $C_{ld}(\text{char}(o_1), \text{char}(o_2))$ considers all the characters that are present before either o_1 or o_2 is generated. If $C_{ld}(\text{char}(o_1), \text{char}(o_2)) = \emptyset$, by Definition 2, we infer $\text{char}(o_2) \prec \text{char}(o_1)$. $\square$

Corollary 3. Given that o_1 is admissible in a reachable state s and o_2 is admissible in $s' = \text{exec}(s, o_1)$, then ET is correct or $o_2' = \text{ET}(o_2, o_1)$ is admissible in s if the effects relation between $\text{char}(o_1)$ and $\text{char}(o_2)$ can be found in ER or can be correctly determined by the rules of $\text{get_er_ET}(o_1, o_2)$.

5 OPERATION INTEGRATION

To execute (integrate) a remote operation o , we often have to transform o against a sequence of operations sq . We denote the process of inclusively transforming o against sq as $\text{ITSQ}(o, sq)$ and that of exclusively transforming o against sq as $\text{ETSQ}(o, sq)$. In this section, we study how to execute remote operations and how to transform sequences.

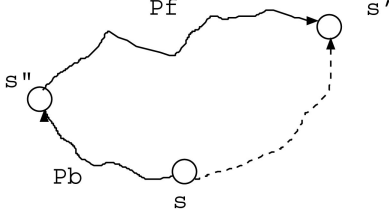


Fig. 2. Different transformation paths from state s to state s' .

5.1 Problem and Analysis

Conceptually, to integrate a remote operation o , we must first compute its admissible form o' relative to the current state and then execute it. This problem can be abstracted in the following way: *Given an operation o generated (admissible) in a given reachable state s , compute its admissible form o' in another reachable state s' , where $s \neq s'$.*

In the abstract sense, computing o' includes two steps: 1) choose or build a proper path (or sequence) P from s to s' and 2) transform o against P . There may exist more than one path from s to s' . As shown in Fig. 2, some paths are unidirectional such that we only need to transform o with some P . Some other paths may be bidirectional in that they first consider a path P_b from s to a significant intermediate state s'' and then a path P_f from s'' to s' .

Specifically, we maintain an operation log HB at every site to record operations in their order of execution. When a remote operation o is received, we integrate o iff all operations that happened before o have been executed locally. Suppose HB can be somehow transposed into $sq_h + sq_c$, where sq_h includes all operations that happened before o and sq_c includes all operations in HB that are concurrent with o . It must be that $s = \text{exec}(s^0, sq_h)$ and $s' = \text{exec}(s^0, HB)$.

Most previous OT algorithms use sq_c as the transformation path from s to s' and compute $o' = \text{ITSQ}(o, sq_c)$, such as adOPTed [14], GOTO [19], [17], SOCT2 [16], and SDT [9], [10]. However, the ordering between concurrent operations in sq_c is not constrained, except the causal order between them. It means that concurrent operations may be ordered arbitrarily at different sites. As a result, their IT functions must verify TP1 and TP2 to achieve convergence [14], which have turned out extremely difficult in practice [17], [20], [22]. Previous works in [12], [15], [20], [22] achieve convergence by enforcing a unique transformation path at all sites and avoid verification of TP2. However, the transformation paths they choose are effectively unconstrained in the same way. That is, arbitrary transformation path is still allowed as in adOPTed, GOTO, SOCT2, and SDT.

In this work, we explore an alternative strategy which avoids arbitrary transformation paths. Instead, we build some special paths out of HB such that our IT and ET can always work correctly. The idea is to first find a backward path P_b from s to s'' and then a forward path P_f from s'' to s' . With these two paths, we first compute an admissible operation $o'' = \text{ETSQ}(o, P_b)$ relative to s'' and then compute the admissible $o' = \text{ITSQ}(o'', P_f)$ relative to s' .

However, our analyses in Section 4 reveal that $\text{IT}(o_1, o_2)$ and $\text{ET}(o_1, o_2)$ do not always work correctly. There are extra conditions in addition to $\text{est}(o_1) = \text{est}(o_2)$ and

$$\text{est}(o_1) = \text{exec}(\text{est}(o_2), o_2),$$

respectively. Intuitively, we somehow transpose sq_h into $sq_{hi} + sq_{hd}$ such that sq_{hi} includes all insertions that happened before o and sq_{hd} includes all deletions that happened before o . Let P_b be sq_{hd} and P_f be $sq_{hd} + sq_c$. We further transpose P_f into $sq_i + sq_d$ such that sq_i only includes insertions and sq_d includes all deletions. As a result, s'' is the state in which all characters deleted by sq_{hd} are recovered. Hence, we can always correctly process $\text{ITSQ}(o'', sq_i + sq_d)$ because no landmark character between insertion o'' and any insertion in sq_i has been deleted, and IT between o'' and any deletion in sq_d is correct. How to build such P_b and P_f and how to ensure the correctness of IT or ET at every step are the remaining problems.

In the following, we discuss how to fulfill the above intuition. Section 5.2 first introduces some notations. In Section 5.3 and 5.4, we study some special sequences that ensure correctness of ITSQ and ETSQ, respectively. Sections 5.5 and 5.6 define some utility functions for transposing sequences and building special sequences. Section 5.7 shows how to integrate remote operations by constructing P_b and P_f .

5.2 Some Notations

Two operations o_1 and o_2 are contextually serialized if $\text{exec}(\text{est}(o_1), o_1) = \text{est}(o_2)$. A sequence of operations $[o_1, o_2, \dots, o_n]$ is contextually serialized if, for any

$$i, 1 \leq i \leq n-1, \text{exec}(\text{est}(o_i), o_i) = \text{est}(o_{i+1}).$$

A contextually serialized sequence is referred to as a sequence. Given a sequence sq , $sq[0]$ refers to the first operation in sq , $sq[1]$ the second, and so forth. Notation $|sq|$ denotes the size of sq or the total number of operations in sq . Notation $sq[i, j]$ denotes the subsequence ranging from the i th to the j th operations in sq , inclusively, where $0 \leq i \leq j < |sq|$.

We generalize function $s' = \text{exec}(s, o)$ to sequences: Given state s^0 and sequence sq , where $s^0 = \text{est}(sq[0])$ and $|sq| = n$, $s^n = \text{exec}(s^0, sq) = \text{exec}(\text{exec}(s^0, sq[0]), sq[1, n-1])$. That is, executing a sequence sq in state s^0 is equivalent to executing $sq[0]$ in s^0 , which results in $s^1 = \text{exec}(s^0, sq[0])$, then executing $sq[1]$ in $s^1 = \text{exec}(s^1, sq[1])$, and so forth.

Given two operation sequences sq_1 and sq_2 , if

$$\text{est}(sq_2[0]) = \text{exec}(\text{est}(sq_1[0]), sq_1),$$

we also say that sq_1 and sq_2 are contextually serialized. For the same reason, operation $sq_1[n-1]$ and sequence sq_2 , and sequence sq_1 and operation $sq_2[0]$, are also contextually serialized. Notation $sq_1 + sq_2$ concatenates two sequences sq_1, sq_2 into one sequence. Two sequences sq_1, sq_2 are effects equivalent, denoted as $sq_1 \sim sq_2$; if executed in the same state, they produce the same result, i.e., $s = \text{est}(sq_1[0]) = \text{est}(sq_2[0])$ and $\text{exec}(s, sq_1) = \text{exec}(s, sq_2)$.

5.3 IT-Safe Operation Sequences

Function $\text{ITSQ}(o, sq)$ requires that o and sq be defined in the same state, or $\text{est}(o) = \text{est}(sq[0])$. It inclusively transforms o against $sq[0], sq[1], \dots, sq[|sq|-1]$ in turn. To ensure the correctness of every IT, we further require that sq is a special sequence, as defined below:

Algorithm 5 $\text{ITSQ}(o, sq)$: o'

```

1:  $o' \leftarrow o$ ;
2: for ( $i = 0$ ;  $i < |sq|$ ;  $i++$ ) do
```

```

3:   $o' \leftarrow \text{IT}(o', sq[i])$ 
4:  if  $o' = \phi$  then
5:    break
6:  end if
7: end for
8: return  $o'$ 

```

Definition 7. A sequence sq is an IT-Safe Operation Sequence (ITSOS) iff for any two operations $sq[i]$ and $sq[j]$, if $\text{type}(sq[i]) = \text{ins}$ and $\text{type}(sq[j]) = \text{del}$, then $i < j$ holds. If $|sq| < 2$, we also say that sq is (trivially) an ITSOS.

An ITSOS sq can be rewritten as $sq_i + sq_d$, where sq_i includes all insertions in sq and sq_d all deletions in sq .

Lemma 13. Given operation o and sequence sq , $o' = \text{ITSQ}(o, sq)$ is admissible if $\text{est}(sq[0]) = \text{est}(o)$ and either 1) $\text{type}(o) = \text{del}$ or 2) $\text{type}(o) = \text{ins} \wedge sq$ is an ITSOS $\wedge$ for any insertion $sq[i]$: $C_{ld}(\text{char}(sq[i]), \text{char}(o)) \subseteq \text{est}(sq[i])$.

Proof. If $\text{type}(o) = \text{del}$, by Lemmas 6 and 8, $\text{IT}(o, sq[i])$ is correct for any operation $sq[i]$. Consider case $\text{type}(o) = \text{ins}$. Because sq is an ITSOS, let $sq = sq_i + sq_d$ and $\text{ITSQ}(o, sq) = \text{ITSQ}(o, sq_i + sq_d)$. By the given condition, for any insertion

$$sq_i[j] : C_{ld}(\text{char}(sq_i[j]), \text{char}(o)) \subseteq \text{est}(sq_i[j]),$$

meaning that all characters in $C_{ld}(\text{char}(sq_i[j]), \text{char}(o))$ must appear in $\text{est}(sq_i[j])$. That is, $\text{pos}(sq_i[j]) = \text{pos}(o)$ if $C_{ld}(\text{char}(sq_i[j]), \text{char}(o))$ is $\emptyset$, or $\text{pos}(sq_i[j]) \neq \text{pos}(o)$ if otherwise. Then, by Lemma 5, when inclusively transforming o against any insertion in sq_i , no landmark character between them has been lost (or deleted). Hence, $\text{IT}(o, sq_i[j])$ is correct for any insertion $sq_i[j]$. By Lemma 7, $\text{IT}(o, sq_d[k])$ is correct for any deletion $sq_d[k]$. $\square$

5.4 ET-Safe Operation Sequences

$\text{ETSQ}(o, sq)$ requires that sq and o be contextually serialized, i.e., $\text{est}(o) = \text{exec}(\text{est}(sq[0]), sq)$. As shown in Algorithm 6, to compute $\text{ETSQ}(o, sq)$, we exclusively transform o against $sq[n-1], sq[n-2], \dots, sq[0]$ in turn. To ensure the correctness of every ET, we further require that sq be a special sequence, as defined below. Due to properties of this sequence, we break the loop of lines 2-7 when some condition is satisfied because it is no longer necessary to call $\text{ET}()$ on the remaining operations in sq . Instead, effects of those operations are excluded directly as in lines 8-14.

Algorithm 6 $\text{ETSQ}(o, sq)$: o'

```

1:   $o' \leftarrow o$ 
2:  for  $(i = |sq| - 1; i \geq 0; i--)$  do
3:     $o' \leftarrow \text{ET}(o', sq[i])$ 
4:    if  $\text{char}(sq[i]) \prec \text{char}(o')$  then
5:      break
6:    end if
7:  end for
8:  for  $(j = i - 1; j \geq 0; j--)$  do
9:    if  $\text{type}(sq[j]) = \text{ins}$  then
10:      $\text{pos}(o') = \text{pos}(o') - 1$ 
11:   else
12:      $\text{pos}(o') = \text{pos}(o') + 1$   $\{\text{// type}(sq[j]) = \text{del}\}$ 
13:   end if

```

```

14: end for
15: return  $o'$ 

```

Definition 8. A given sequence sq is an ET-Safe Operation Sequence (ETSOS) if, for any i, j , where $0 \leq i < j \leq n - 1$ and $|sq| = n$, either 1) $\text{char}(sq[i]) \prec \text{char}(sq[j])$ or 2) $\text{char}(sq[i]) = \text{char}(sq[j])$ and $\text{type}(sq[i]) = \text{ins}$ and $\text{type}(sq[j]) = \text{del}$. If $|sq| < 2$, we also say that sq is (trivially) an ETSOS.

Operations in an ETSOS sq are ordered by their effects relation, except that inverse operations are ordered by their causal order, i.e., a deletion o_d that deletes the character inserted by some o_i appears after o_i . As a simple example, given an initial state $s^0 = \text{"b"}$ and three operations executed in tandem: $o_1 = \text{ins}(0, \text{'a'})$, $o_2 = \text{del}(1)$, and $o_3 = \text{ins}(1, \text{'c'})$. The effects relation is $\text{'a'} \prec \text{'b'} \prec \text{'c'}$ and $[o_1, o_2, o_3]$ is an ETSOS.

Lemma 14. Given operation o and an ETSOS sq , if there exists some $sq[i]$, $0 \leq i \leq |sq| - 1$, such that $\text{char}(sq[i]) \prec \text{char}(o)$, then $\text{char}(sq[j]) \prec \text{char}(o)$ for all j : $0 \leq j \leq i - 1$.

The correctness of Lemma 14 is straightforward by the definition of ETSOS. The result can simplify the implementation of $\text{ETSQ}(o, sq)$. When o encounters some $sq[i]$ in $\text{ETSQ}()$, if $\text{char}(sq[i]) \prec \text{char}(o)$, the effects relation between o and any operation $sq[j]$, where $j < i$, must be $\text{char}(sq[j]) \prec \text{char}(o)$. As a result, the remaining operations in sq can be correctly excluded from o without $\text{get_er_ET}()$. This property is used in $\text{ETSQ}()$ to avoid calling $\text{ET}()$, as shown in lines 4-6 and 8-14. In addition, if o and some operation $sq[i]$ have the same effect character, namely, $\text{char}(o) = \text{char}(sq[i])$, or o deletes a character inserted by some $sq[i]$, it does not make sense to perform this and the remaining ET's due to causality. The system is halted in this case. Other algorithms that call $\text{ETSQ}()$ will prevent this from happening.

Lemma 15. Given operation o and sequence sq , where $|sq| = n$ and $\text{est}(o) = \text{exec}(\text{est}(sq[0]), sq)$, let C_{sq} denote the set of characters that appear in $\text{est}(sq[0])$ and sq , that is,

$$C_{sq} = \bigcup_{i=0}^{n-1} \{\text{char}(sq[i])\} \cup \text{est}(sq[0]).$$

Then, $o' = \text{ETSQ}(o, sq)$ is admissible if 1) sq is an ETSOS and 2) $C_{ld}(\text{char}(o), \text{char}(sq[i])) \subseteq C_{sq}$ for any $sq[i] \rightarrow o$.

Proof. Due to Lemmas 9 through 12 and Corollary 3 in Section 4.2, the only case under which $\text{ET}(o, sq[i])$ can be unsafe for some $sq[i]$ is when $\text{pos}(sq[i]) = \text{pos}(o)$, $sq[i] \rightarrow o$, $\text{type}(sq[i]) = \text{del}$, and $\text{type}(o) = \text{ins}$. Hence, we only need to prove the correctness of $\text{ET}(o, sq[i])$ under this unsafe condition. We prove this by induction.

Base: Consider $\text{ET}(o, sq[n-1])$. First, by definition of ETSOS, we know $\text{char}(sq[k]) \prec \text{char}(sq[n-1])$ for $0 \leq k < n - 1$. By condition

$$C_{ld}(\text{char}(o), \text{char}(sq[n-1])) \subseteq C_{sq},$$

for any $sq[k]$, where $0 \leq k < n - 1$, it is impossible to have $\text{char}(sq[n-1]) \prec \text{char}(sq[k]) \prec \text{char}(o)$. Second, under the above-identified unsafe condition of ET,

$\text{type}(sq[n-1]) = \text{del}$, $\text{type}(o) = \text{ins}$, $sq[n-1] \rightarrow o$, and $\text{pos}(sq[n-1]) = \text{pos}(o)$, it is impossible to have any c in $\text{est}(o)$ such that $\text{char}(sq[n-1]) \prec c \prec \text{char}(o)$. Otherwise, we would have $\text{pos}(sq[n-1]) < \text{pos}(o)$, which contradicts $\text{pos}(sq[n-1]) = \text{pos}(o)$. Since

$$\text{est}(o) = \text{exec}(\text{est}(sq[0]), sq)$$

and $C_{sq} = \bigcup_{i=0}^{n-1} \{\text{char}(sq[i])\} \cup \text{est}(sq[0])$, with the above two conclusions combined, we infer that there is no character c in C_{sq} such that $\text{char}(sq[n-1]) \prec c \prec \text{char}(o)$. Therefore, we conclude $C_{id}(\text{char}(o), \text{char}(sq[n-1])) = \emptyset$, or there exists at least one character c such that $\text{char}(o) \prec c \prec \text{char}(sq[n-1])$. In either case, by the definition of relation $\prec$, we get $\text{char}(o) \prec \text{char}(sq[n-1])$. Hence, $\text{ET}(o, sq[n-1])$ is correct.

Induction: Assume that processing of $\text{ETSQ}(o, sq[i+1, n-1])$ is correct. Let o' be the execution form of o relative to $\text{est}(sq[i+1])$. By Lemma 14 and Algorithm 6, at the time $\text{ET}(o', sq[i])$ is processed, o' must have been exclusively transformed with all operations from $sq[n-1]$ to $sq[i+1]$, inclusively. Hence, $\text{char}(o) \prec \text{char}(sq[k])$ must hold, where $i+1 \leq k \leq n-1$. Otherwise, the ET process should have stopped for some k . Since $C_{id}(\text{char}(o), \text{char}(sq[i])) \subseteq C_{sq}$, similar to the base case, there must not exist c such that $\text{char}(sq[i]) \prec c \prec \text{char}(o)$, or we would have $\text{pos}(sq[i]) < \text{pos}(o)$, which contradicts $\text{pos}(sq[i]) = \text{pos}(o)$. Then, by the definition of relation $\prec$, no matter whether or not there exists c such that $\text{char}(o) \prec c \prec \text{char}(sq[i])$, we get $\text{char}(o) \prec \text{char}(sq[i])$. Hence, $\text{ET}(o, sq[i])$ is correct. $\square$

5.5 Transposing Sequences

$\text{Transpose}(o_2, o_1)$ is to transpose two contextually serialized operations o_2 and o_1 such that in the output o'_1 is contextually serialized before o'_2 . Since o_2 is contextually serialized before o_1 , we first do $o'_1 = \text{ET}(o_1, o_2)$ such that o'_1 and o_2 are defined in the same state, and then do $o'_2 = \text{IT}(o_2, o'_1)$ such that o'_2 is defined in state $\text{exec}(\text{est}(o'_1), o'_1)$. Note, if o_1 depends on o_2 , i.e., $\text{type}(o_2) = \text{ins}$, $\text{type}(o_1) = \text{del}$, $o_2 \rightarrow o_1$, and $\text{pos}(o_1) = \text{pos}(o_2)$, we know $\text{char}(o_1) = \text{char}(o_2)$ and return $\langle o_1, o_2 \rangle$.

Algorithm 7 $\text{Transpose}(o_2, o_1): \langle o'_1, o'_2 \rangle$

```

1:  $\{\text{determined as in get\_er\_ET}\}$ 
2: if  $\text{char}(o_1) = \text{char}(o_2)$  then
3:   return  $\langle o_1, o_2 \rangle$ 
4: else
5:    $o'_1 \leftarrow \text{ET}(o_1, o_2)$ 
6:    $o'_2 \leftarrow \text{IT}(o_2, o'_1)$ 
7:   return  $\langle o'_1, o'_2 \rangle$ 
8: end if
```

Algorithm 8 $\text{TransposeOSq}(sq, o): \langle o', sq' \rangle$

```

1:  $o' \leftarrow o; sq' \leftarrow sq$ 
2: for  $(i = |sq| - 1; i \geq 0; i--)$  do
3:    $\langle o', sq'[i] \rangle \leftarrow \text{Transpose}(sq'[i], o')$ 
4: end for
5: return  $\langle o', sq' \rangle$ 
```

The precondition of $\text{TransposeOSq}(sq, o)$ is that sq is contextually serialized before o . The output is such that o' is

contextually serialized before sq' . In general, all the transposition functions in this paper produce effects-equivalent sequences. Specifically, $[o'_1, o'_2] \sim [o_2, o_1]$ in $\text{Transpose}(o_2, o_1)$, $o' + sq' \sim sq + o$ in $\text{TransposeOSq}(sq, o)$, $sq_h + sq_c \sim sq$ in $\text{TransposePreCon}(o, sq)$, $sq_i + sq_d \sim sq$ in $\text{TransposeInsDel}(sq)$, and $sq' \sim sq$ in $\text{BuildETSOS}(sq)$.

Algorithm 9 $\text{TransposePreCon}(o, sq): \langle sq_h, sq_c \rangle$

```

1:  $sq_h \leftarrow \emptyset; sq_c \leftarrow \emptyset$ 
2: for  $(i = 0; i < |sq|; i++)$  do
3:   if  $sq[i] \parallel o$  then
4:      $sq_c \leftarrow sq_c + sq[i]$ 
5:   else
6:      $\langle o_i, sq_c \rangle \leftarrow \text{TransposeOSq}(sq_c, sq[i])$ 
7:      $sq_h \leftarrow sq_h + o_i$ 
8:   end if
9: end for
10: return  $\langle sq_h, sq_c \rangle$ 
```

Functions $\text{TransposePreCon}()$ and $\text{TransposeInsDel}()$ have similar structures. $\text{TransposePreCon}(o, sq)$ is adopted from [16], [19]. It transposes sq into $sq_h + sq_c$, such that sq_h , all operations that happened before o , are contextually serialized before sq_c , all those concurrent with o . It scans sq from left to right and appends every $sq[i]$ that is concurrent with o to sq_c . For every $sq[i]$ that happened before o , it first transposes $sq[i]$ and sq_c and then appends $sq[i]$ to sq_h . Similarly, $\text{TransposeInsDel}(sq)$ transposes sq such that all insertions (sq_i) are contextually serialized before all deletions (sq_d).

Algorithm 10 $\text{TransposeInsDel}(sq): \langle sq_i, sq_d \rangle$

```

1:  $sq_i \leftarrow \emptyset; sq_d \leftarrow \emptyset$ 
2: for  $(i = 0; i < |sq|; i++)$  do
3:   if  $\text{type}(sq[i]) = \text{del}$  then
4:      $sq_d \leftarrow sq_d + sq[i]$ 
5:   else
6:      $\langle o_i, sq_d \rangle \leftarrow \text{TransposeOSq}(sq_d, sq[i])$ 
7:      $sq_i \leftarrow sq_i + o_i$ 
8:   end if
9: end for
10: return  $\langle sq_i, sq_d \rangle$ 
```

5.6 Building ET-Safe Sequences

Given a sequence sq , function $\text{BuildETSOS}(sq)$ incrementally builds an ETSOS sq' . Initially, sq' only includes $sq[0]$. Then, each time a new element $sq[i]$ is added into sq' . It transposes $sq[i]$ with operations in sq' from right to left until some $sq'[j]$ is found (line 12) such that $\text{char}(sq'[j]) \prec \text{char}(o)$, which is exactly the condition defined in Lemma 14. Under this case, o is inserted after $sq'[j]$ (line 13). The relation between $\text{char}(o)$ and other operations in the rest of sq is known and directly recorded in the local effects relation ER (line 10). If the condition never appears, as in lines 20-22, then o is added to the head of sq' .

Lemma 16. *Given an admissible sequence sq defined in the initial system state s^0 , which preserves the causal order of operations, $sq' = \text{BuildETSOS}(sq)$ is correct, i.e., sq' is an ETSOS and every operation in sq' is admissible.*

Proof. Examine the process of building an ETSOS sq' from sq : Each time we add a new operation $sq[i]$ into the

partial result sq' that was built from $sq[0, i-1]$, sequence sq' is scanned from right to left to find the right position to insert $sq[i]$. Note that each operation in sequence sq' actually represents a state transition. Hence, to insert $sq[i]$ before some position $sq'[j]$, we have to transpose $sq'[j, i-1]$ and $sq[i]$. That is, the correctness of BuildETSOS() depends on the correctness of Transpose().

In function $\text{Transpose}(o_1, o_2)$, ET is always called before IT. If $o'_1 = \text{ET}(o_1, o_2)$ is correct, then the relation of $\text{char}(o_1)$ and $\text{char}(o_2)$ is known, which ensures the correctness of $\text{IT}(o_2, o'_1)$. Hence, we only need to prove the correctness of $\text{ETSQ}(sq[i], sq'[j, i-1])$. This can be further reduced to prove that the conditions of Lemma 15 can be satisfied before calling BuildETSOS(). We prove this by induction.

Algorithm 11 BuildETSOS(sq): sq'

```

1: if  $|sq| < 1$  then
2:   return  $sq' \leftarrow sq$ 
3: end if
4:  $sq' \leftarrow [sq[0]]$ 
5: for  $(i = 1; i < |sq|; i++)$  do
6:    $o \leftarrow sq[i]$ 
7:    $flag \leftarrow \text{false}$ 
8:   for  $(j = |sq'| - 1; j \geq 0; j--)$  do
9:     if  $flag = \text{true}$  then
10:      record  $\text{char}(sq'[j]) \prec \text{char}(o)$  into  $ER$ 
11:     else  $\{\text{// effects relation determined as in get\_er\_ET}\}$ 
12:       if  $\text{char}(sq'[j]) \prec \text{char}(o)$  then
13:          $sq' \leftarrow sq'[0, j] + o + sq'[j+1, |sq'| - 1]$ 
14:          $flag \leftarrow \text{true}$ 
15:       else
16:          $\langle o, sq'[j] \rangle \leftarrow \text{Transpose}(sq'[j], o)$ 
17:       end if
18:     end if
19:   end for
20:   if  $flag = \text{false}$  then
21:      $sq' \leftarrow o + sq'$ 
22:   end if
23: end for
24: return  $sq'$ 

```

Base: Consider $\text{ET}(sq[1], sq[0])$. If $sq[1] \parallel sq[0]$, then $\text{ET}(sq[1], sq[0])$ is correct because the effects relation between concurrent operations has been recorded. Otherwise, it must be $sq[0] \rightarrow sq[1]$ by causality. Because $\text{gst}(sq[0]) = s^0$ and there is no other operation, we have $C_{id}(\text{char}(sq[0]), \text{char}(sq[1])) \subseteq C_{sq}$. Hence, $\text{ET}(sq[1], sq[0])$ is correct.

Induction: Suppose sq' is the ETSOS correctly built from $sq[0, i-1]$. Now, check how $sq[i]$ is added into sq' . Let $C_{sq'} = \bigcup_{j=0}^{i-1} (\text{char}(sq'[j])) \cup s^0$. Since $sq'[0]$ is defined on s^0 and all operations that happened before $sq[i]$ are in sq' , it is obvious that $C_{id}(\text{char}(sq[i]), \text{char}(sq'[j])) \subseteq C_{sq'}$ for any $0 \leq j < |sq'|$ and $sq'[j] \rightarrow sq[i]$. By Lemma 15, $\text{ETSQ}(sq[i], sq')$ is correct. $\square$

5.7 The Integration Procedure

The top-level control algorithm executes local and remote operations. For best responsiveness, any local operation is executed and appended to HB once it is generated. Any

remote operation o is queued until it is causally-ready, i.e., all operations that happened before o are in HB . Suppose the initial state is s^0 , the current state is s' , and o is generated in state s . Then, function $\text{Integrate}(o, HB)$ is called to derive an o' such that o' is admissible in s' . If $o' = \phi$, it is discarded. Otherwise, o' is executed and appended to HB . Apparently, HB is a contextually serialized sequence.

Algorithm 12 $\text{Integrate}(o, HB)$: o'

```

1:  $\langle sq_h, sq_c \rangle \leftarrow \text{TransposePreCon}(o, HB)$ 
2: if  $sq_c = \emptyset$  then
3:   return  $o' \leftarrow o$ 
4: end if
5: if  $\text{type}(o) = \text{del}$  then
6:   return  $o' \leftarrow \text{ITSQ}(o, sq_c)$ 
7: else  $\{\text{// type}(o) = \text{ins}\}$ 
8:    $sq'_h \leftarrow \text{BuildETSOS}(sq_h)$ 
9:    $\langle sq_{hi}, sq_{hd} \rangle \leftarrow \text{TransposeInsDel}(sq'_h)$ 
10:   $sq_{hdc} \leftarrow \text{BuildETSOS}(sq_{hd} + sq_c)$ 
11:   $\langle sq_i, sq_d \rangle \leftarrow \text{TransposeInsDel}(sq_{hdc})$ 
12:   $o'' \leftarrow \text{ETSQ}(o, sq_{hd}) \{\text{// } P_b = sq_{hd}\}$ 
13:   $o' \leftarrow \text{ITSQ}(o'', sq_i + sq_d) \{\text{// } P_f = sq_i + sq_d\}$ 
14:  return  $o'$ 
15: end if

```

In $\text{Integrate}(o, HB)$, we first call $\text{TransposePreCon}(o, HB)$ to transpose HB into two sequences: sq_h , which includes all operations that happened before o , and sq_c , which includes all operations that are concurrent with o . Due to causality, we have $os = \text{gst}(o) = \text{exec}(s^0, sq_h)$.

If sq_c is empty, it means $s' = \text{gst}(o)$. Then, o is returned as it is (lines 2-3) and will be executed in s' directly. If sq_c contains concurrent operations, we must incorporate the effects of those operations into o to get its execution form o' in s' . By the above analyses, we cannot perform $\text{ITSQ}(o, sq_c)$ until the preconditions defined in Lemma 13 are satisfied. If $\text{type}(o) = \text{del}$, $\text{ITSQ}(o, sq_c)$ is guaranteed to be correct (lines 5-6).

However, if $\text{type}(o) = \text{ins}$, we must first construct P_b and P_f , then compute $o'' = \text{ETSQ}(o, P_b)$, and, finally, $o' = \text{ITSQ}(o'', P_f)$. Sequence P_b is constructed in lines 8-9: We first build an ETSOS sq'_h from sq_h , the sequence of all operations that happened before o , and then transpose sq'_h into two subsequences, sq_{hi} and sq_{hd} , such that sq_{hi} includes all insertions that happened before o and sq_{hd} includes all deletions that happened before o . Then, sq_{hd} is just the transformation path P_b we need.

Next, we construct P_f as in lines 10-11: We first build an ETSOS sq_{hdc} from $sq_{hd} + sq_c$ and then transpose sq_{hdc} into two sequences, sq_i and sq_d , such that sq_i includes all the insertions and sq_d all the deletions. Sequence P_f is just $sq_i + sq_d$, the concatenation of sq_i and sq_d .

Theorem 2. $o' = \text{Integrate}(o, HB)$ is admissible in state s' .

Proof. The assertion is easily verified if $\text{type}(o) = \text{del}$. When $\text{type}(o) = \text{ins}$, the correctness of $\text{Integrate}()$ relies on four steps:

1. $\text{TransposePreCon}()$ in line 1,
2. the construction of P_b in lines 8-9 and P_f in lines 10-11,
3. $\text{ETSQ}(o, P_b)$ in line 12, and
4. $\text{ITSQ}(o'', P_f)$ in line 13.

First, in $\text{TransposePreCon}(o, HB)$, the cause-effect order between operations in HB are preserved. It can be shown that the Transpose algorithm is called only between concurrent operations. The reason is that, due to the definition of Algorithm 9, all operations in sq_c are concurrent with o . Each time we call $\text{TransposeOSQ}(sq_c, sq[i])$ or $\text{Transpose}(sq_c[j], sq[i])$ for some $sq_c[j]$, due to causality preservation, $sq[i]$ must have happened before o , or $sq[i] \rightarrow o$. In addition, since $sq[i]$ is stored after $sq_c[j]$ in HB due to Algorithm 9, we cannot have $sq[i] \rightarrow sq_c[j]$ due to causality. Hence, we have either $sq_c[j] \rightarrow sq[i]$ or $sq_c[j] \parallel sq[i]$. The former case cannot happen because, if otherwise, we would have $sq_c[j] \rightarrow sq[i] \rightarrow o$ and $sq_c[j] \rightarrow o$ by transitivity, which contradicts $sq_c[j] \parallel o$. Therefore, $sq_c[j] \parallel sq[i]$ must be true and the effects relation between them must have been recorded in the local memory ER , which ensures the correctness of Transpose of ET/IT at each step.

Second, by Lemma 16, $sq'_h = \text{BuildETSOS}(sq_h)$ in line 8 is correct. As a result, operations in sq'_h are ordered by their effects relation and, additionally, if deletion o_d deletes the character inserted by some o_i , then o_i always appears before o_d . Consequently, in the algorithm of $\text{TransposeInsDel}(sq'_h)$ in line 9, for any insertion $sq'_h[i]$ and any deletion $sq_{hd}[j]$ in $\text{Transpose}(sq_{hd}[j], sq'_h[i])$, the case of $\text{char}(sq'_h[i]) = \text{char}(sq_{hd}[j])$ will never happen. Due to the effects relation recorded in $\text{BuildETSOS}(sq_h)$, all ET/IT in $\text{TransposeInsDel}(sq'_h)$ can be correctly processed.

Rewrite HB as $sq_h + sq_c = sq_{hi} + sq_{hd} + sq_c$. When building an ETSOS out of $sq_{hd} + sq_c$, no character has been lost (or deleted) in the definition state of $sq_{hd} + sq_c$ or state $s'' = \text{exec}(s^0, sq_{hi})$ due to the execution of sequence sq_{hi} in s^0 . Hence, the step of $sq_{hdc} = \text{BuildETSOS}(sq_{hd} + sq_c)$ in line 10 is correct, the proof of which resembles that of Lemma 16. Similarly, $\text{TransposeInsDel}(sq_{hdc})$ in line 11 is also correct.

Third, consider $o'' = \text{ETSQ}(o, P_b)$, where $P_b = sq_{hd}$. It can be shown from the algorithm of $\text{TransposeInsDel}()$ that P_b preserves the effects relation-based order between operations as a result of $\text{BuildETSOS}()$. Hence, P_b itself is also an ETSOS, although all its operations are deletions. Let $C_{P_b} = \bigcup_{i=0}^{|P_b|-1} \{\text{char}(P_b[i])\} \cup \text{est}(P_b[0])$. Since P_b only contains deletions, for any $P_b[i] \rightarrow o$, when processing $\text{ET}(o, P_b[i])$, condition $C_{ld}(\text{char}(o), \text{char}(P_b[i])) \subseteq C_{P_b}$ must hold. By Lemma 15, o'' is admissible in state $s'' = \text{exec}(s^0, sq_{hi}) = \text{est}(P_b[0])$.

Fourth, consider $o' = \text{ITSQ}(o'', P_f)$, where $P_f = sq_i + sq_d$ is built from sequence $sq_{hd} + sq_c$. Conceptually, o' must incorporate the effects of sq_c and, additionally, sq_{hd} because the effects of sq_{hd} have been excluded from o'' earlier. Sequence P_f is an ITSOS because it is a concatenation of an insertion sequence sq_i and a deletion sequence sq_d , which were correctly computed earlier. Consequently, because sequences sq_{hi} and sq_i do not contain any deletions, the preconditions of Lemma 13 are satisfied. Therefore, $o' = \text{ITSQ}(o'', P_f)$ is admissible in the current state $s' = \text{exec}(s^0, HB) = \text{exec}(s'', P_f)$. $\square$

6 AN EXAMPLE

As shown in Fig. 3, four sites start from the same initial state $s^0 = "1."$ Site 1 performs $o_1 = \text{ins}(1, 'b')$. Concurrently, site 2

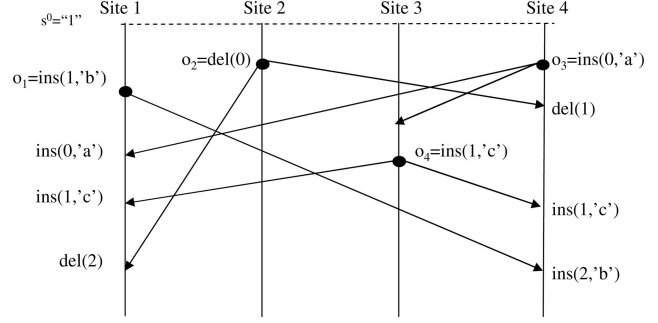


Fig. 3. Four sites start from state "1" and converge in "acb."

performs $o_2 = \text{del}(0)$ and site 4 performs $o_3 = \text{ins}(0, 'a')$. After o_3 is executed, site 3 performs $o_4 = \text{ins}(1, 'c')$. We have $o_1 \parallel o_2$, $o_2 \parallel o_3$, $o_1 \parallel o_3$, $o_3 \rightarrow o_4$, $o_1 \parallel o_4$, and $o_2 \parallel o_4$. Suppose the execution order at site 4 is o_3, o_2, o_4, o_1 . For space reasons, we only consider how o_1 and o_4 are integrated at site 4 to illustrate the working of our approach.

When o_4 arrives at site 4, $HB = [o_3, o'_2]$, where $o'_2 = \text{IT}(o_2, o_3) = \text{del}(1)$ and the current state

$$s_4^2 = \text{exec}(s^0, [o_3, o'_2]) = "a."$$

To compute o'_4 relative to s_4^2 , we first construct P_b , which includes all deletions that happened before o_4 . Hence, P_b is empty by definition. Then, we construct P_f , which is initially $sq_{hd} + sq_c = sq_c = [o'_2]$. Therefore,

$$o'_4 = \text{ITSQ}(o_4, [o'_2]) = \text{ins}(1, 'c').$$

After o'_4 is executed, the state of site 4 becomes $s_4^3 = "ac."$

When o_1 arrives, $HB = [o_3, o'_2, o'_4]$. Similarly, P_b is empty. Hence, P_f is initially HB itself because all the operations are concurrent with o_1 . We transpose P_f into $sq_i + sq_d$ such that $sq_i = [o_3, o_4]$ and $sq_d = [o'_2]$, where $o'_2 = \text{del}(2)$. Then, $o'_1 = \text{ITSQ}(o_1, [o_3, o_4, o'_2]) = \text{ins}(2, 'b')$. After o'_1 is executed, the final state of site 4 becomes $s_4^4 = "acb."$

7 CONCLUSIONS

The presented work makes the following two significant contributions: First, it proposes a novel framework (called CR due to its correctness criteria, causality preservation, and operation effects relation preservation) for interpreting and developing OT algorithms. In the theory part, it formalizes a new constraint, effects relation preservation, for constraining convergence in interactive groupware applications. Compared to the intention preservation constraint in the state-of-the-art framework CCI [20], it is well-formalized and subject to correctness proofs. The effects relation is naturally defined when operations are generated, except some "artificial" tie-breaking policies for handling boundary cases.

In the technical approach part, CR reveals an alternative way of developing OT algorithms, which follows three main steps: First, it defines an application-specific effects relation between objects for constraining the execution of operations. Second, it formulates a set of transformation functions and provides sufficient conditions for them to satisfy the defined effects relation. Third, it finds a special transformation path for computing an admissible operation and ensuring the preconditions of the transformation functions.

To evaluate the proposed *CR* framework, we exemplify a novel OT algorithm (called landmark-based transformation or LBT), which is testimony to the above design guidelines. Specifically, the transformation functions in LBT only use basic information such as operation type, position parameter, and site id. The concepts of IT/ET safe operation sequences reveal practicable heuristics for constructing the special transformation paths. As demonstrated by the design of the LBT algorithm, the *CR* framework simplifies the design of transformation functions because it no longer demands that transformation functions (IT) work in all possible cases. That is, it theoretically frees the difficult TP2 condition that is established in [14] and well-accepted in the literature [16], [17], [19].

However, more work is necessary to evaluate these ideas in a wider range of application domains. The proposed framework and algorithms seem generally applicable for optimistic consistency control on linear data structures. In future research, we plan to extend this work to support tree-based data structures [4] and string operations [20]. Another possible direction is to support selective undo [17] such that any executed operation can be undone at any time. Conceptually, computing an admissible operation in group undo is an inverse process of that in group do. The presented work seems to provide valuable insights into this problem. In addition, this work defines a set of sufficient conditions for IT/ET as a means to develop OT algorithms and prove their correctness. In future work, it would also be interesting to study necessary conditions and even sufficient and necessary conditions.

ACKNOWLEDGMENTS

The authors thank the associate editor, Dr. Philip K. McKinley, and the anonymous expert referees for their insightful and constructive review comments, which significantly improved the readability, technical quality, and presentation of this paper. This work was supported in part by the US National Science Foundation under CAREER award IIS-0133871.

REFERENCES

- [1] B.R. Badrinath and K. Ramamritham, "Semantics-Based Concurrency Control: Beyond Commutativity," *ACM Trans. Database Systems*, vol. 17, no. 1, pp. 163-199, Mar. 1992.
- [2] P. Bellini, P. Nesi, and M.B. Spinu, "Cooperative Visual Manipulation of Music Notation," *ACM Trans. Computer-Human Interaction*, vol. 9, no. 3, pp. 194-237, Sept. 2002.
- [3] P.A. Bernstein, V. Hadzilacos, and N. Goodman, *Concurrency Control and Recovery in Database Systems*. Addison-Wesley, 1987.
- [4] A.H. Davis, C. Sun, and J. Lu, "Generalizing Operational Transformation to the Standard General Markup Language," *Proc. ACM Conf. Computer-Supported Cooperative Work (CSCW '02)*, pp. 58-67, Nov. 2002.
- [5] P. Dewan, R. Choudhary, and H. Shen, "An Editing-Based Characterization of the Design Space of Collaborative Applications," *J. Organizational Computing*, vol. 4, no. 3, pp. 219-240, 1994.
- [6] C.A. Ellis and S.J. Gibbs, "Concurrency Control in Groupware Systems," *Proc. ACM SIGMOD Conf. Management of Data*, pp. 399-407, 1989.
- [7] S. Greenberg and D. Marwood, "Real-Time Groupware as a Distributed System: Concurrency Control and Its Effect on the Interface," *Proc. ACM Conf. Computer-Supported Cooperative Work (CSCW '94)*, pp. 207-217, Oct. 1994.
- [8] M.J. Knister and A. Prakash, "A Framework for Undoing Actions in Collaborative Systems," *ACM Trans. Computer-Human Interaction*, vol. 1, no. 4, pp. 295-330, Dec. 1994.
- [9] D. Li and R. Li, "Ensuring Content and Intention Consistency in Real-Time Group Editors," *Proc. 24th IEEE Int'l Conf. Distributed Computing Systems (ICDCS '04)*, pp. 748-755, Mar. 2004.
- [10] D. Li and R. Li, "Preserving Operation Effects Relation in Group Editors," *Proc. ACM Conf. Computer-Supported Cooperative Work (CSCW '04)*, pp. 457-466, Nov. 2004.
- [11] D. Li and R. Li, "An Approach to Ensuring Consistency in Peer-to-Peer Real-Time Group Editors," *Computer Supported Cooperative Work: J. Collaborative Computing*, Oct. 2005.
- [12] R. Li, D. Li, and C. Sun, "A Time Interval Based Consistency Control Algorithm for Interactive Groupware Applications," *Proc. IEEE Int'l Conf. Parallel and Distributed Systems (ICPADS '04)*, pp. 429-436, July 2004.
- [13] M. Raynal and M. Singhal, "Logical Time: Capturing Causality in Distributed Systems," *Computer*, pp. 49-56, Feb. 1996.
- [14] M. Ressel, D. Nitsche-Ruhland, and R. Gunzenhäuser, "An Integrating, Transformation-Oriented Approach to Concurrency Control and Undo in Group Editors," *Proc. ACM Conf. Computer-Supported Cooperative Work (CSCW '96)*, pp. 288-297, Nov. 1996.
- [15] H. Shen and C. Sun, "Flexible Notification for Collaborative Systems," *Proc. ACM Conf. Computer-Supported Cooperative Work (CSCW '02)*, pp. 77-86, Nov. 2002.
- [16] M. Suleiman, M. Cart, and J. Ferrié, "Concurrent Operations in a Distributed and Mobile Collaborative Environment," *Proc. IEEE Int'l Conf. Data Eng. (ICDE '98)*, pp. 36-45, Feb. 1998.
- [17] C. Sun, "Undo as Concurrent Inverse in Group Editors," *ACM Trans. Computer-Human Interaction*, vol. 9, no. 4, pp. 309-361, Dec. 2002.
- [18] C. Sun and D. Chen, "Consistency Maintenance in Real-Time Collaborative Graphics Editing Systems," *ACM Trans. Computer-Human Interaction*, vol. 9, no. 1, pp. 1-41, Mar. 2002.
- [19] C. Sun and C. Ellis, "Operational Transformation in Real-Time Group Editors: Issues, Algorithms, and Achievements," *Proc. ACM Conf. Computer-Supported Cooperative Work (CSCW '98)*, pp. 59-68, Dec. 1998.
- [20] C. Sun, X. Jia, Y. Zhang, Y. Yang, and D. Chen, "Achieving Convergence, Causality-Preservation, and Intention-Preservation in Real-Time Cooperative Editing Systems," *ACM Trans. Computer-Human Interaction*, vol. 5, no. 1, pp. 63-108, Mar. 1998.
- [21] D. Sun, S. Xia, C. Sun, and D. Chen, "Operational Transformation for Collaborative Word Processing," *Proc. ACM Conf. Computer-Supported Cooperative Work (CSCW '04)*, pp. 162-171, Nov. 2004.
- [22] N. Vidot, M. Cart, J. Ferrié, and M. Suleiman, "Copies Convergence in a Distributed Realtime Collaborative Environment," *Proc. ACM Conf. Computer-Supported Cooperative Work (CSCW '00)*, pp. 171-180, Dec. 2000.
- [23] W.E. Weihl, "Commutativity-Based Concurrency Control for Abstract Data Types," *IEEE Trans. Computers*, vol. 37, no. 12, pp. 1488-1504, Dec. 1988.
- [24] H. Yu and A. Vahdat, "Design and Evaluation of a Conit-Based Continuous Consistency Model for Replicated Services," *ACM Trans. Computer Systems*, vol. 20, no. 3, pp. 239-282, Aug. 2002.



Rui Li is a PhD candidate in the Department of Computer Science, Texas A&M University. His research interests include groupware, user interfaces, and distributed systems. His PhD dissertation focuses on operational transformation-based consistency control in group editors. He is a student member of the IEEE.



Du Li received the PhD degree from the University of California, Los Angeles in 2000. He is currently an assistant professor in the Department of Computer Science, Texas A&M University. His research interests include groupware, computer-supported cooperative work, distributed systems, and software engineering. He is a member of the IEEE and the IEEE Computer Society.